НИЯУ МИФИ

Бинарные уязвимости

Модуль 2. Динамический анализ. Введение в бинарные уязвимости

План занятия

НИЯУ МИФИ

Динамический анализ. Введение в бинарные уязвимости

1. Динамический анализ. Основные способы
2. ltrace
3. strace
4. GDB + pwndbg
5. pwntools
6. Введение в бинарные уязвимости
7. Типичные кейсы: переписывание строки, указателя, переполнение int
8. Итоги

Динамический анализ. Основные способы

Основные подходы к реверсу

Статический анализ

1. Получаем ассемблерный или исходный код
2. Исследуем код без запуска

Динамический анализ

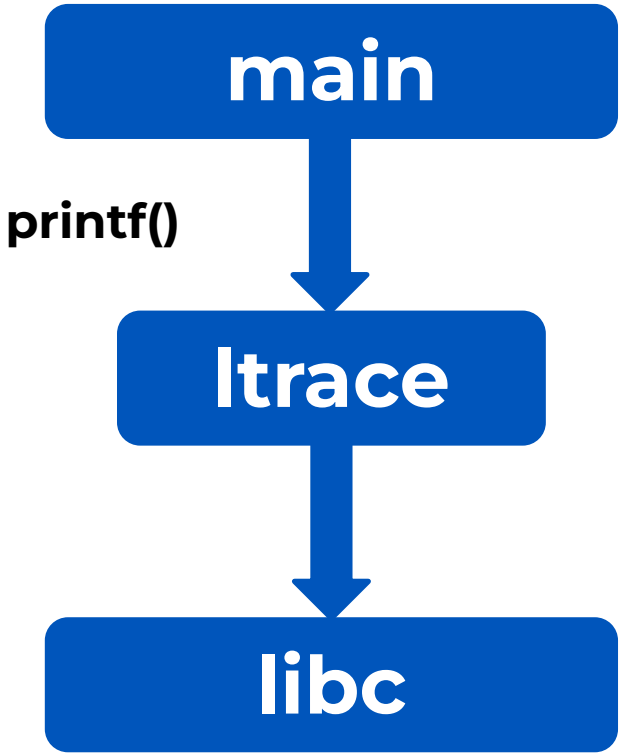
1. Запускаем программу в отладчике
2. Исследуем, что происходит на каждом шаге (инструкции, память)

Основные способы

1. **ltrace** – перехват вызовов библиотечных функций
2. **strace** – перехват системных вызовов
3. **GDB + pwndbg** - отладка

ltrace

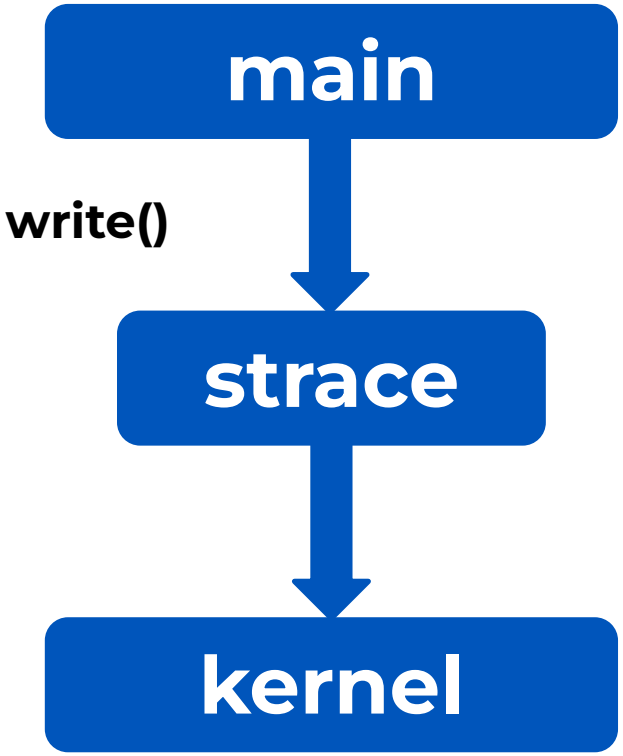
1. *ltrace ./main*
2. *-e 'f1 + f2'* – только f1, f2
3. *-e '!f1'* – исключить f1
4. *-l lib* – библиотека
5. *-s N* – длины строк
6. *-r* – коды возврата
7. *-f* – дочерние процессы



[Проект OpenNet: MAN ltrace (1) Команды и прикладные программы пользовательского уровня (FreeBSD и Linux)]

strace

1. *strace ./main*
2. *-f* – дочерние процессы
3. *-e trace=read,write* – определённые сисколы
4. *-s N* – максимальная длина строк
5. *-T* – время выполнения сисколов
6. *-o trace.log* – записать в файл



strace(1) - Linux manual page

Отладка

GDB

- Основной консольный отладчик в Linux
- Позволяет ставить точки останова, выполнять программу по шагам, смотреть память и дизассемблированный код
- Если программа с отладочными символами, будет виден исходный код

pwndbg

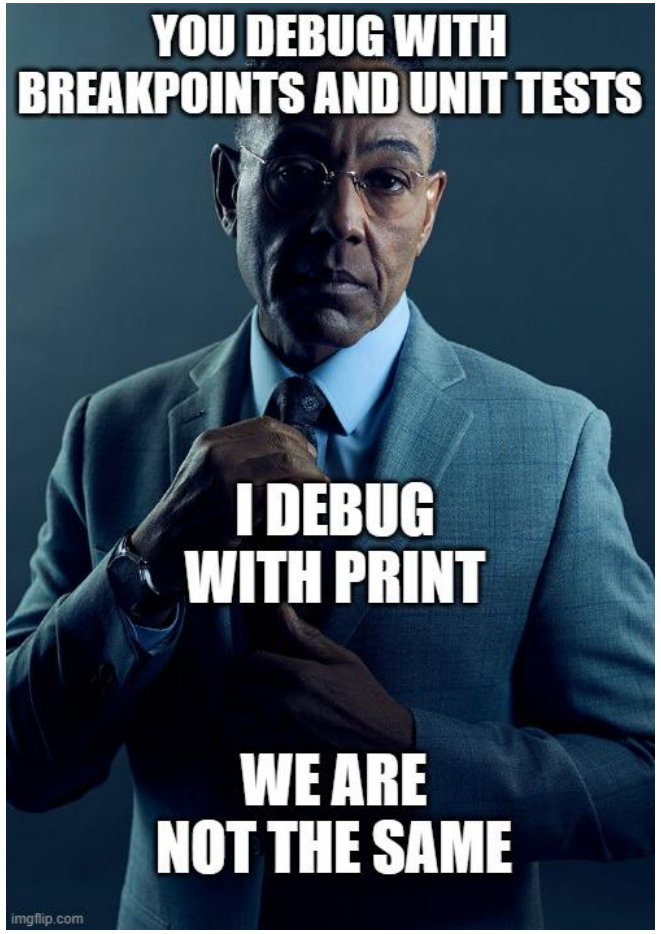
- Плагин для GDB
- Делает анализ нагляднее и удобнее
- Добавляет новые функции

Запуск и подготовка к отладке

1. *gdb ./main*
2. gdb –args ./main secret
3. set args secret
4. show args
5. run / r (далее можно написать аргументы командной строки)
6. quit / q

Точки останова (breakpoints)

1. Программные
   - Инструкция запоминается и заменяется на int3 – исключение
   - Отладчик обрабатывает исключение и восстанавливает инструкцию
   - <u>Неограниченное кол-во, любой код</u>
2. Аппаратные
   - Использует debug-регистры процессора **DR0-DR7**
   - 4 брейкпоинта, не модифицируют код
3. На область памяти (watchpoints)



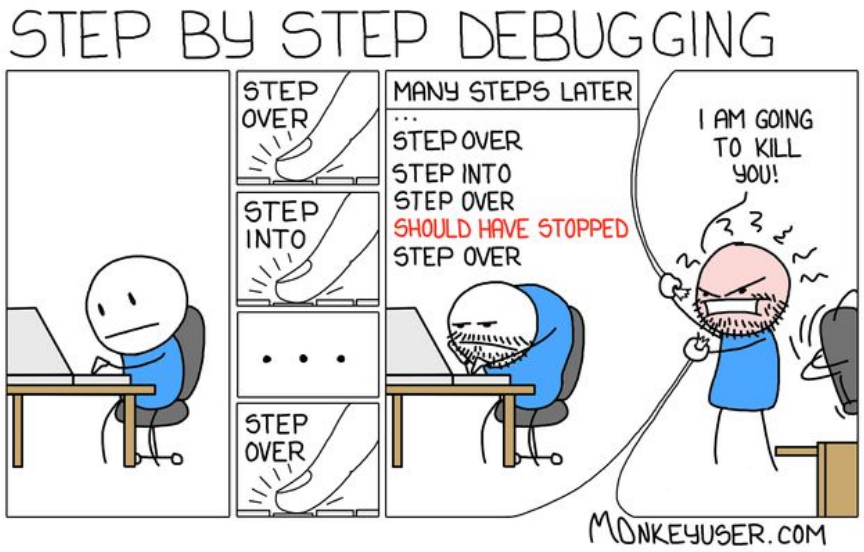
Точки останова (breakpoints)

1. *break / b main*
2. *b main.c:10*
3. *b *0x52525252*
4. *info break / i b*
5. *delete* – удалить все
6. *delete 1* – удалить первый
7. *enable 1 / disable 1* – активировать / деактивировать
8. *condition 1 argc == 5* – условие срабатывания

They can debug with breakpoints and watch instead of prints and logs..

Управление выполнением

1. *next / n* – следующая строка
2. *ni* – следующая ассемблерная инструкция
3. *step / s* – шаг с заходом
4. *si* – следующая инструкция с заходом
5. *continue / c* – продолжить выполнение
6. *finish* – выполнить текущую функцию до возврата
7. *until 10* – выполнить до 10 строки
8. *start* – выполнить до entrypoint



Просмотр исходного кода

1. *list / l*
2. list main
3. list 1,20
4. info source
5. directory src/

Переменные, выражения, память

1. *print / p argc*
2. *print &argc*
3. *print argv[1]*
4. *set var argc = 4*
5. *x /[n][f][u] <адрес>*

- `/`: Format specifiers
- `[n]`: Number of units to display
- `[f]` : Format
  - `x`: Hexadecimal
  - `d`: Decimal
  - `u`: Unsigned Decimal
  - `o`: Octal
  - `t`: Binary
  - `a`: Address
  - `i`: Instruction (disassembly)
  - `c`: Character
  - `s`: String

- `[u]`: Unit size
  - `b`: Byte
  - `h`: Hafword
  - `w`: Word
  - `g`: Giant word

Наблюдение и автоматизация

1. *watch x* – на запись
2. *rwatch x* – на чтение
3. *awatch x* – на чтение и запись
4. *commands 1*
    1. *print x*
    2. *end*

pwntools

Установка библиотеки

pwntools - библиотека для автоматизации pwn-задач и работы с бинарными файлами

pip install pwntools

*from pwn import **

Запуск и подключение

p = process('./binary') - локальный процесс

p = remote('192.168.52.52', 1337) - удалённый процесс
p = remote('ctf.example.com', 5000)

Отправка данных

p.send(data) - отправить байты
p.sendline(data) - отправить + \n
p.sendafter(s, data) - подождать строку s, затем отправить
p.sendlineafter(s, data)

Примеры:
p.sendline(b"test")
p.sendlineafter(b"Input:", payload)
*p.send(b"A" * 16)*

Чтение данных

p.recv(n) - прочитать n байт
p.recvline() - прочитать строку до \n
p.recvuntil(s) - читать до строки s
p.recvall() - прочитать всё до EOF
p.interactive() - передать управление пользователю

Примеры:
data = p.recvline()
p.interactive()

Упаковка данных

p32(addr) - адрес
p32(addr, endian='big')

p64(addr) - little-endian
p64(addr, endian='big') - big-endian

u32(data)
u32(data, endian=’big’)

u64(data) - байты в число
u64(data, endian=’big’)

Работа с ELF

elf = ELF('./binary') - загрузка бинаря

elf.symbols['main'] - адрес функции
elf.got['puts'] - адрес GOT-записи
elf.plt['puts'] - адрес PLT-записи
elf.address - базовый адрес

Введение в бинарные уязвимости (PWN)

Что такое PWN?

1. Эксплуатация уязвимостей в скомпилированных программах
2. **Цели обычно:** выполнить произвольный код, получить shell, повысить привилегии
3. **Основа** - небезопасная работа с памятью, undefined behaviour

Язык C

1. Компилируемый
2. Предоставляет полный контроль над системой
3. Вся ответственность на разработчике
4. Все данные - байты в памяти
5. Доступ к данным по указателям (адресам)

Адресное пространство

Разные части программы находятся в разных частях памяти:

1. **code (.text)** - инструкции
2. **data (.data, .bss)** - глобальные переменные
3. **stack** - локальные переменные, адреса возврата
4. **heap** - динамическая память
5. **lib** - библиотеки (libc...)

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
    0x555555554000     0x555555556000 r-xp     2000 0      /home/kp/Downloads/good/babystack
    0x555555755000     0x555555756000 r--p     1000 1000   /home/kp/Downloads/good/babystack
    0x555555756000     0x555555757000 rw-p     1000 2000   /home/kp/Downloads/good/babystack
    0x7ffff79e4000     0x7ffff7bcb000 r-xp   1e7000 0      /lib/x86_64-linux-gnu/libc-2.27.so
    0x7ffff7bcb000     0x7ffff7dcb000 ---p   200000 1e7000 /lib/x86_64-linux-gnu/libc-2.27.so
    0x7ffff7dcb000     0x7ffff7dcf000 r--p     4000 1e7000 /lib/x86_64-linux-gnu/libc-2.27.so
    0x7ffff7dcf000     0x7ffff7dd1000 rw-p     2000 1eb000 /lib/x86_64-linux-gnu/libc-2.27.so
    0x7ffff7dd1000     0x7ffff7dd5000 rw-p     4000 0
    0x7ffff7dd5000     0x7ffff7dfc000 r-xp    27000 0      /lib/x86_64-linux-gnu/ld-2.27.so
    0x7ffff7fcf000     0x7ffff7fd1000 rw-p     2000 0
    0x7ffff7ff7000     0x7ffff7ffa000 r--p     3000 0      [vvar]
    0x7ffff7ffa000     0x7ffff7ffc000 r-xp     2000 0      [vdso]
    0x7ffff7ffc000     0x7ffff7ffd000 r--p     1000 27000  /lib/x86_64-linux-gnu/ld-2.27.so
    0x7ffff7ffd000     0x7ffff7ffe000 rw-p     1000 28000  /lib/x86_64-linux-gnu/ld-2.27.so
    0x7ffff7ffe000     0x7ffff7fff000 rw-p     1000 0
    0x7ffffffde000     0x7ffffffff000 rw-p    21000 0      [stack]
0xffffffffff600000 0xffffffffff601000 r-xp     1000 0      [vsyscall]
```

Си - не memory-safe

1. Ввод без контроля длины: **gets(), scanf(“%s”)**
2. Копирование строк: **strcpy(), strcat()**
3. Форматный вывод: **printf(buf)**
4. ...

Функции доверяют программисту и контролируют границы, длину и тд.
Ошибка превращается в memory corruption

Типичные кейсы:

1. Переписывание строки
2. Переписывание указателя
3. Переполнение int

Переписывание строки

string.c

```c
// gcc -o string string.c -fno-stack-protector -no-pie
#include <stdio.h>
#include <stdlib.h>

struct Data {
    char name[4];
    char command[16];
} data = { "", "echo Hello" };

int main(void)
{
    puts("Input:");
    gets(data.name); // небезопасно
    printf("You entered: %s\n", data.command);
    system(data.command);
    return 0;
}
```

Переписывание указателя

func.c

```c
// gcc -o func func.c -fno-stack-protector -no-pie -g
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void hello() {
    puts("Hello");
}

void secret() {
    system("cat flag.txt");
}

struct Data {
    char buf[16];
    void (*fp)();
};

int main() {
    struct Data d = { "", hello };
    gets(d.buf);
    d.fp();
    return 0;
}
```

Переполнение int

int.c

```c
// gcc -o int int.c -no-pie -g -fwrapv
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

int main() {
    int balance = 2147483600;
    int add;

    puts("=== CENTRAL COUNTING SYSTEM ===");
    puts("Enter increment:");
    scanf("%d", &add);

    balance += add;

    if (balance < 0) {
        puts("SYSTEM ERROR: integer overflow detected");
        system("cat flag.txt");
    } else {
        printf("Current balance: %d\n", balance);
    }

    return 0;
}
```

Решение задач

Полезные материалы

1. Шпаргалка полезных команд GDB / Хабр
2. GDB Advanced Debugging Skills | Axura
3. https://pwndbg.re/stable/commands/
4. pwn.spbctf.ru/tasks
5. pwn.college
6. play.picoctf.org
7. Yet Another Pwntools Cheatsheet | 🐕.🪦

Спасибо
за внимание!